Approval-Based Elections in the 1D Euclidean Domain

Jan Sznajd

Supervisor: prof. dr hab. inż. Piotr Faliszewski

Institute of Computer Science
AGH University of Science and Technology

This dissertation is submitted for the degree of
Master of Science

May 2021

Contents

1 Introduction **1**

1.1 Preference Restrictions 2

1.2 Contributions 2

2 Preliminaries **4**

2.1 Basic Definitions 4

2.2 Preference Restrictions 5

2.2.1 Euclidean Preference Restrictions 6

2.3 WIP: Methodology/Techniques 7

2.3.1 Integer Linear Program 7

3 Voter/Candidate Range Profile Detection **8**

3.1 Profile Structure Detection 8

3.2 Integer Linear Program 9

3.3 WIP: Consecutive Ones Detection — ILP vs SAT 11

4 Voter/Candidate Range Profiles **12**

4.1 Domain analysis 12

4.2 How many VCR profiles are not VR neither CR? 12

4.2.1 Experiment 1 — Small Profiles 12

4.2.2 Experiment 2 — Large Profiles 13

4.3 Transforming VCR (NCOP) Profile into VR or CR 17

4.3.1 Experiment 3 — Profile Transformation 18

5 Control Problem **19**

5.1 Overview 19

5.1.1 AV-CC-DV for Single Peaked (VR = CI = row C1P = $r_c = 0$) . . . 19

5.1.2 AV-CC-DV for Candidate Range = VI = column C1P = $r_v = 0$ . . . 20

5.2 AV-CC-DV for the VCR profiles 20

5.2.1 Sketch Proof - CR case 21
5.2.2 Sketch Proof - VR case 22
5.2.3 Sketch Proof - VCR (not C1P) case 24
5.3 AV-CC-AV for the VCR profiles 27

6 Evaluation **28**
6.1 Conclusions 28
6.2 Future Work 28

Bibliography **29**

Chapter 1

Introduction

Elections are a ubiquitous element of our society in one form or another. For example, societies need to choose members of parliaments, judges have to pick winners of sports competitions. However, elections are not only applicable to politics or competitions. In more day-to-day examples of elections, teams have to schedule meeting times, video streaming platforms need to provide recommended movies list to the users. Each of these choices (problems) may require different type of an election and different strategy of selecting the winner (or winners). The differences come from the underlying properties and objectives of the choice. For example, in presidential elections, we want to choose the candidate supported by the majority of the voters. It is not the case for the parliamentary elections, where the goal is to select members of the parliament in such a way that the opinions of the voters are represented proportionally. In the meeting scheduling example, we will try to pick the time, such that, we maximize the number of participants, even though, the selected time might not be optimal for anyone (e.g., early morning). Another key distinction between the given examples, is whether a given election has just one winner, or it leads to selecting multiple candidates. More formally, a committee election is the process of choosing a fixed-size subset of candidates (i.e., the committee), based on the voters' preferences. Voters' ballots (preferences) can be represented in various different ways. (WIP: Rozróżnić Single vs Multi winner) In this work we are going to focus on approval elections, that is, on elections where each voter provides a subset of candidates that he or she approves of. Another notable type of committee elections is ordinal model, where the voters order the candidates from the most to the least preferred. See the work of Faliszewski et al. (2017), which provides the overview and discusses various committee election types, and for the extensive survey of approval model and current state of the research in this area, we point to Lackner and Skowron (2020).

Winner determination is one of the core problems in computational social choice theory, however not the only one. We need to mention control problems, it refers to certain ways of

manipulating elections. In short, we try to make a given candidate win (or lose), by changing the structure of an election, that is adding or deleting voters or candidates. Other interesting problems, that we are not going to explore in this work, are manipulation, bribery, justified representation and many others. (WIP: Cytacje?)

1.1 Preference Restrictions

Many of the mentioned above social choice problems are known to be computationally hard. We can deal with this complexity using various methods, more standard ones, like approximate (random or deterministic) or parametrized algorithms. An alternative approach to these classical solutions is introducing restrictions on preference domains, in other words, we restrict the structure of voters ballots (preferences).

There is a successful and active line of research in this area. Beginning with a notion of *single-peaked (SP)* (Black (1958); Arrow (1951)) and *single-crossing (SC)* (Mirrlees (1971); Roberts (1977)). Arriving to much more recent work, Barberà and Moreno (2011) introduced *top-monotonicity*, a generalization of SP and SC restrictions. These models provided a basis for numerous and promising results in circumventing the computational complexity in a variety of problems. (Walsh (2007); Faliszewski et al. (2009); Skowron et al. (2015); Brandt et al. (2015); Elkind et al. (2016))

However, in the context of this work we need to emphasize a survey by Elkind and Lackner (2015). Unlike most of the above-mentioned examples, which considered the ordinal model of preferences, the survey explored well known preference restrictions in the setting of approval preferences. Besides that, it introduced several new one-dimensional domain restriction models such as *Voter Interval (VI)* and *Candidate Interval (CI)*, both based on the *Dichotomous Euclidean (DE)* restriction. (WIP: Opisać formalnie DE?) Faliszewski (2021) improved this model by making it more versatile, due to using additional radius parameter and providing N-dimensional definition. (WIP: Jaki był aspekt uzupełnienia modeli euklidesowych? Czy tu jest za wcześnie na mówienie o promieniu?) Calling the new model *Voter Candidate Range (VCR)*, and later they analyzed two-dimensional case and briefly one-dimensional, for committee election problems.

1.2 Contributions

In this work we focus on one-dimensional Voter Candidate Range model, and our main contributions are following:

1. We provide detection algorithm for VCR property, based on the ILP. Additionally, we extended it to detect VR and CR properties, and compare it with existing solutions.
2. We analyze VCR profiles domain extensively, and compare it to other well known models.
3. We propose control problem polynomial time algorithm for the VCR model, expanding on the solutions for CR and VR models, from Magiera and Faliszewski (2014) and Faliszewski et al. (2009) respectively.
4. (WIP: Napisać trochę więcej i podać numery rozdziałów/sekcji)

We do not discuss the related work right now, instead we do it throughout this thesis, as the discussion will greatly benefit from the added context and introduced definitions in the subsequent chapters.

Chapter 2

Preliminaries

First, in Section 2.1, we introduce basic definitions and core concepts, together with simple examples. Later, in Section 2.2, we discuss preference restrictions, and how can we leverage them to design better algorithms, for many of the computational social choice problems. Finally, in Section 2.3, we explain some computational methods (tricks) we have used in this work, that we believe are not necessarily straightforward.

2.1 Basic Definitions

Approval Elections

An approval-based election is a pair $E = (C, V)$, where $C = \{c_1, ..., c_M\}$ is a set of candidates and $V = \{v_1, ..., v_N\}$ is a set of voters. We use M and N for the number of candidates and voters respectively. For each voter $v \in V$, his or hers preferences are represented as a set of approved candidates $A(v)$, we call it an *approval set*. Furthermore, for each candidate $c \in C$, we define $V(c) = \{v \in V \mid c \in A(v)\}$, a set of voters who approve candidate c.

In practice, we often use an additional, simplified notation of representing an election. We redefine A as a two-dimensional binary matrix, with N rows and M columns, where value of $A[i][j]$ is (1) if i-th voter approves of j-th candidate, and (0) otherwise. In other words, each row of the matrix is a vector representation of a given voter's approval set. We refer to it as an *approval matrix* or a *preference matrix*. It will always be clear from the context which notation we are using. (WIP: Jak definiować macierze?)

Example 1. *(WIP: Opis wyborów + figura) Following the example from Figure 2.1 ...*

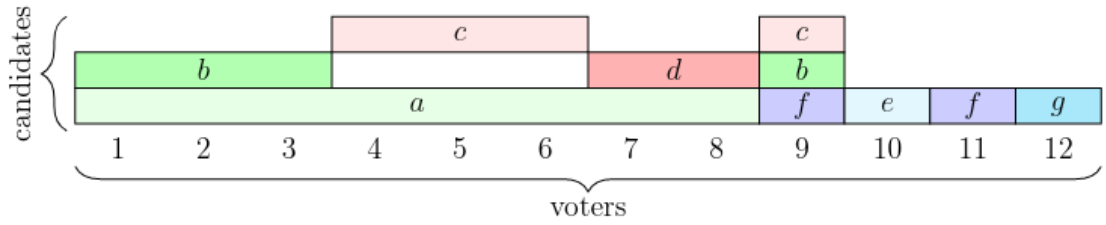

Figure 2.1 Temporary Image - Example of An Approval Election Lackner and Skowron (2020)

Multiwinner Committee Selection Rules

Voting rules are algorithms used to, select a committee (winners) — fixed size subset of the candidates, based on the voters preferences, for a given election. Each rule has different properties and must be selected according to the goal of the election. As this is not the main focus of this work we point to Faliszewski et al. (2017), for a detailed overview of multiwinner voting rules.

Nonetheless, we introduce the most basic voting rule for the approval model, simply called *Approval Voting (AV)*, introduced by Brams and Fishburn (1983).

Definition 1 (Multi-Winner Approval Voting (AV))**.** *For a given election and a size of committee k, AV rule selects the k candidates with the highest approval score, which, for each candidate c, is the number of voter who approve of c. (WIP: Poprawić definicje, dodac oznaczenia matematyczne)*

Example 2. *(WIP: Opis wyborów z poprzedniego przykładu + kto wygra) Continuing with the election from Example 1*

2.2 Preference Restrictions

Many computational social choice problems such as winner determination, or control problem are known to be computationally hard. To deal with it, we can use approximate or parametrized algorithms The alternative approach is to introduce restrictions on the preference domains. In the last few years this area of research advanced a lot, providing multiple efficient algorithms for the popular committee selection rules, and other problems.

2.2.1 Euclidean Preference Restrictions

Dichotomous Euclidean (DE)

Voter Candidate Range (VCR)

Given an election $E = (C,V)$, we say that the voters have *1D-Euclidean* preferences if for every agent (voter or candidate) there exists a point x_a and nonnegative real value radius r_a such that: $\forall\, c \in C\ \forall\, v \in V : c \in A(v) \Leftrightarrow \sqrt{(x_c - x_v)^2} \leq r_c + r_v$ N We can easily think of it as: the point x_a represents ideal position in the 1-dimensional space of opinions. The radius can be seen as candidate's charisma or voter's willingness to compromise.

Candidate Range (CR)

We say that P satisfies CR if candidates in C can be ordered so that each of the sets v_i forms an interval of that ordering. CR is special case of VCR, where we require all the voters to have radii equal to zero

Voter Range (VR)

We say that P satisfies VR if the voters in V can be reordered so that for every candidate c_i the voters that approve c form an interval of that ordering. VR is a special case of VCR, where we require all the candidates to have radii equal to zero

Voter Interval (VI) and Candidate Interval (CI)

Faliszewski (2021) showed that CR and VR are equivalent to respectively VI and CI introduced by Elkind and Lackner (2015).

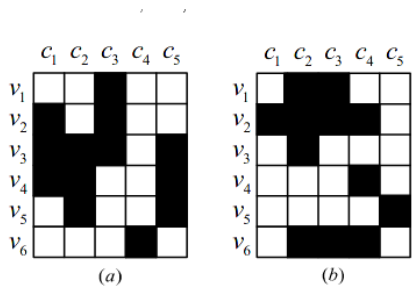

Figure 2.2 CR and VR Temporary image

Larger Domain

(WIP: Zmienić na mój przykład i opisać dlaczego nie vr i nie cr)

The VCR model has larger domain than the VR and CR combined. For example, election with candidate set $C = \{a,b,c,d\}$ and voters with approval sets $\{a,b\}$, $\{b,c\}$, $\{b,d\}$, and $\{a,b,c,d\}$ is VCR, but neither VR nor CR.

2.3 WIP: Methodology/Techniques

2.3.1 Integer Linear Program

Integer linear programming (ILP) is a common approach to solve NP-hard problems, furthermore it can be used effectively for other problems as well. (WIP: czy zdanie od furthermore ... jest potrzebne) To define a computational problem as an integer linear program, we have to create a system of linear inequalities (constraints) (WIP: Sprawdzic czy na pewno inequalities == constraints) and optionally a linear function to optimize, alternatively we can just check feasibility of the constraints. Next to obtain a solution we have to use an optimization solver. Popular choices are Gurobi or CPLEX which are state-of-the-art general purpose solvers.

Chapter 3

Voter/Candidate Range Profile Detection

In this chapter we present our algorithms for deciding if a given approval profile has the VCR property.

3.1 Profile Structure Detection

To take advantage of the restrictions defined in Section **??**, we need to design detection algorithms. They can be used to check whether given election profiles satisfy particular structural properties. For example, to check the VR (CR) property it is enough to check for the *consecutive ones property* in the preference matrix.

Definition 2. *A binary matrix (i.e., a matrix consisting of zeros and ones) has the consecutive ones property (C1P) if its columns be permuted in such a way that for each row the ones are consecutive, i.e., the 1s form an interval.*

In this work we refer to the term *C1P* more broadly, using it interchangeably, for an analogous scenario where we permute the rows and look for the consecutive 1s in every column in a given matrix. Booth and Lueker (1976) provided a linear-time C1P detection algorithm based on the PQ-Tree data structure.

Proposition 1 (Elkind and Lackner (2015))**.** *Deciding whether an approval profile satisfies VR or CR is possible in $O(m \cdot n)$ time.*

Proof. Let $C = \{c_1, ..., c_m\}$ and $V = \{v_1, ..., v_n\}$, we create a $n \times m$ matrix A (a_{ij}): for each $i \in [n]$ and $j \in [m]$: $a_{ij} = 1$ if $c_j \in A(v_i)$ and 0 otherwise. We can see that for the VR property, permuting the columns of matrix A so that ones form an interval in each row is equivalent to permuting candidates so that the set of candidates approved by each voter forms an interval. For the CR case, we just need to transpose the matrix created for the VR. □

Our focus lies on the one dimensional VCR property. Having preference matrix as input we have to provide for each agent (voter/candidate) a point on the real line and a nonnegative real value representing radius, so that these parameters satisfy the VCR property.

3.2 Integer Linear Program

An algorithm based on ILP solving is an excellent choice for the first attempt at designing a detection algorithm. It provides an efficient and relatively simple solution. Furthermore, it can be used as a fast benchmark for validating future solutions. Below we provide an ILP for the VCR profile recognition, together with additional preliminary steps.

Transforming the Definition of VCR into Linear Constraints

Before formulating an ILP for detecting VCR profiles, we have to redefine the definition of VCR in order to make it appropriate for an ILP solver. The definition is based on the euclidean distance which cannot be directly represented in an ILP. Equation 3.2 shows transformation of the VCR definition into linear inequalities.

As a short reminder from Section 2.2.1, for the candidates C and voters V, 1D-VCR is defined as:

$$\forall\, c \in C\, \forall\, v \in V : c \in A(v) \Leftrightarrow |x_c - x_v| \leq r_c + r_v \tag{3.1}$$

Where x_a and r_a represent agent's point on the real line and their radius respectively.

(WIP: W jaki sposób mogę zwięźle, bez ponownego wpropadzania wszystkich oznaczeń, przypomieć definicję? O ile w ogóle jest to potrzebne...)

$$\begin{aligned} &|x_c - x_v| \leq r_c + r_v \\ &\iff -(r_c + r_v) \leq x_c - x_v \leq r_c + r_v \\ &\iff x_c - x_v \leq r_c + r_v \;\; \vee \;\; -(x_c - x_v) \leq r_c + r_v \end{aligned} \tag{3.2}$$

(WIP: Poprawić formatowanie, jak?)

The constraint obtained from the equation 3.2 is not suitable yet for an ILP because it contains logical *OR* condition which again cannot be directly represented in an ILP. To transform it into linear form, we apply commonly called *BigM* technique, as shown below.

$$
\begin{aligned}
x_c - x_v &\leq r_c + r_v + M(1 - z_1) \\
-(x_c - x_v) &\leq r_c + r_v + M(1 - z_2) \\
z_1 + z_2 &\geq 1 \\
z_1, z_2 &\in \{0, 1\}
\end{aligned}
\tag{3.3}
$$

First, we have to introduce a large constant M (larger than $max(|x|)$) and two binary variables z_1 and z_2. Now if any of the z_i is 0 then the corresponding inequality will be true, and the additional constraint $(z_1 + z_2 \geq 1)$ guarantees that at least one of the z_i will be 1.

(WIP: Trochę namieszane wytłumaczenie, bo tak naprawdę problem z *OR* jest tylko dla wariantu if $c \notin A(v)$, czyli $x_c - x_v + M(1 - z) \geq r_c + r_v + 1$ ...)

The BigM technique is common enough, for most solvers to provide additional built-in mechanisms for applying it. Using them is usually recommended, as it allows a solver for better internal optimization. In Gurobi solver we use *indicator constraint*, which allows one to create versatile constraints in form of: $z = f \rightarrow a^T x \leq b$. The constraint states that if the binary indicator variable z is equal to f, where $f \in \{0, 1\}$, then the linear constraint $a^T x \leq b$, where a and b are data coefficients vectors and x represents the decision variables, should hold. On the other hand, if $z = 1 - f$, the linear constraint may be violated.

(WIP: BigM jest używane do reprezentacji różnych 'funkcji', nie tylko *OR*, np. *max*, *abs*, etc., czy też o tym wspomnieć?)

Detection Algorithm

Final formulation of the ILP for recognising VCR property in a given election profile is shown in Figure 3.4. The program accepts preference matrix A as input, and if solution exists it outputs every agent's position x_a and radius r_a. Additionally, we use two binary variables p and q, required by the indicator constraints.

$$
\begin{aligned}
&\textbf{check feasibility}: \\
&\quad \textbf{subject to}: \\
&\qquad x_a \in \mathbb{R} && \forall\, \mathrm{a} \in C \cup V \\
&\qquad r_a \in \mathbb{R}^{0+} && \forall\, \mathrm{a} \in C \cup V \\
&\qquad p_{cv} \in \{0,1\} && \forall\, \mathrm{c} \in \mathrm{C}\, \forall\, \mathrm{v} \in V \\
&\qquad q_{cv} \in \{0,1\} && \forall\, \mathrm{c} \in \mathrm{C}\, \forall\, \mathrm{v} \in V \\
&\quad \text{if } c \in A(v): \\
&\qquad x_c - x_v \leq r_c + r_v \\
&\qquad -(x_c - x_v) \leq r_c + r_v \\
&\quad \text{if } c \notin A(v): \\
&\qquad p_{cv} == 1 \rightarrow x_c - x_v \geq r_c + r_v + 1 \\
&\qquad q_{cv} == 1 \rightarrow -(x_c - x_v) \geq r_c + r_v + 1 \\
&\qquad p_{cv} + q_{cv} >= 1
\end{aligned}
\tag{3.4}
$$

3.3 WIP: Consecutive Ones Detection — ILP vs SAT

Chapter 4

Voter/Candidate Range Profiles

In this section we explore the VCR profiles domain and describe some of its properties. The main goal is to compare the size of the domain with other well known domains such as VR and CR.

4.1 Domain analysis

WIP: Najpierw Napisać dlaczego / po co to robimy. Dopiero potem co / w jaki sposób. Chcemy sprawdzić czy warto wymyślać algorytmy VCR i czy doemna jest znacząco większa od VR / CR

We have performed several experiments, in each of them we check how many profiles have the VCR property but not VR neither CR. For the first one we focus on the small size elections, and for the second experiment we employ several strategies to generate random, larger profiles with the VCR property.

4.2 How many VCR profiles are not VR neither CR?

4.2.1 Experiment 1 — Small Profiles

For the small profiles we did a complete search of the domain. That is we generated every possible profile of a given size and checked if it has the VCR property. We checked 6 dimensions of the elections with the following sizes, having M candidates and N voters, $(M,N) \in \{(3,3),(4,4),(5,5),(4,6),(6,4)\}$. The results as seen in Figure 4.1 show that VCR domain is relatively large, however only limited number of profiles are neither VR nor CR.

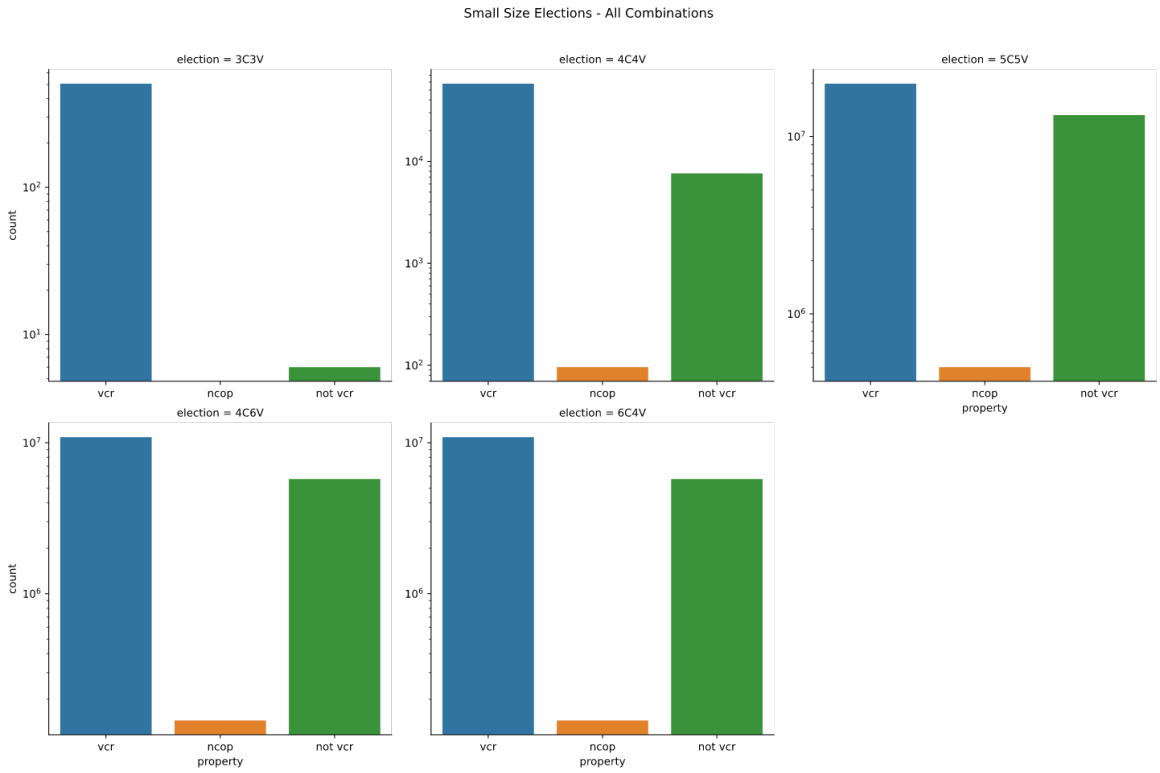

Figure 4.1 Temporary Image - Aggregated Statistics For Small Profiles

4.2.2 Experiment 2 — Large Profiles

For the larger size of the profiles we generated a suitable number of random VCR profiles and again checked how many of them are neither CR nor VR (*NCOP*), and additionally, VR and CR (*COP*), VR and not CR, CR and not VR. We employed several strategies for generating the profiles. The first parameterized category is a statistical distribution used to pick the points representing voters and candidates. The second one is the choice of radius parameter. We describe the categories in detail below. For each parameter pair we generated 1000 elections with 20 candidates and 20 voters, then we repeated the experiment for the profiles of 40 candidates and 40 voters.

Position Parameter

We consider 4 ways of generating the points, representing the positions of the candidates and the voters.

1. The *Uniform Candidates, Uniform Voters* model, where the points for both, the candidates and the voters are selected uniformly at random from the $[-10, 10]$ range.

2. The *Asymmetric Gauss Candidates, Asymmetric Gauss Voters* model, where we take 70% of points from a Gaussian distribution with mean value -3 and standard deviation 1.8, and the remaining 30% come from a Gaussian with mean 3 and the same standard deviation. This model is inspired by the 2 dimensional model from the work of Faliszewski (2021). The goal is to simulate the society with a large majority, and a minority with the opposite opinion.

3. The *Uniform Candidates, Asymmetric Gauss Voters* model, it is a combination of the previous two models, where the candidate's points are selected uniformly at random, and for the voters we use asymmetric Gaussian model. We use the same parameters for both of the distributions.

4. The *Asymmetric Gauss Candidates, Uniform Voters* model, it is mirror version of the previous model, with the distributions swapped.

Radius Parameter

In this category we consider 5 ways of generating the radii. We split the methods between the candidates and the voters, similarly to picking the positions.

1. The *Gauss Candidates, Gauss Voters* model, where we choose the radii from the Gaussian distribution with mean value 1.5 and standard deviation 0.5 We use the same distribution for the candidates and the voters.

2. The *Small Uniform Candidates, Gauss Voters* model. We pick radii for the candidates uniformly at random from the range $[0.7, 1.2]$, and for the voters we use the same Gaussian, as in the previous model ($Gauss(1.5, 0.5)$).

3. The *Gauss Candidates, Small Uniform Voters* model. In this model we swap the distributions used in the previous one, that is, the candidates are picked from the Gaussian and the voters from the uniform distributions.

4. The *Small Uniform Candidates, Small Uniform Voters* model. We pick radii for both the candidates and the voters, uniformly at random from the range $[0.7, 1.2]$.

5. The *Large Uniform Candidates, Large Uniform Voters* model. We pick radii for both the candidates and the voters, uniformly at random from the range $[0, 3]$.

At first, it might be surprising why we don't include a constant radius, as one of the parameters. That is because, then all the profiles would be VR and CR (*COP*), as we show in Proposition 2.

Proposition 2. *If a profile P satisfies VCR and every agent (candidate/voter) in the profile, has the same constant radius, then P satisfies both VR and CR properties.*

Proof. Because. (WIP: Dokończyć, R=const –> 2 razy większy R kandydata i R=0 głosujacego, i vice versa) □

Results

We present the visualization of our example election profiles, and how are they impacted by the generation parameters in Figure 4.2. Later we analyze aggregated results of the experiment. Most of the conclusions we draw are inline with the basic intuition.

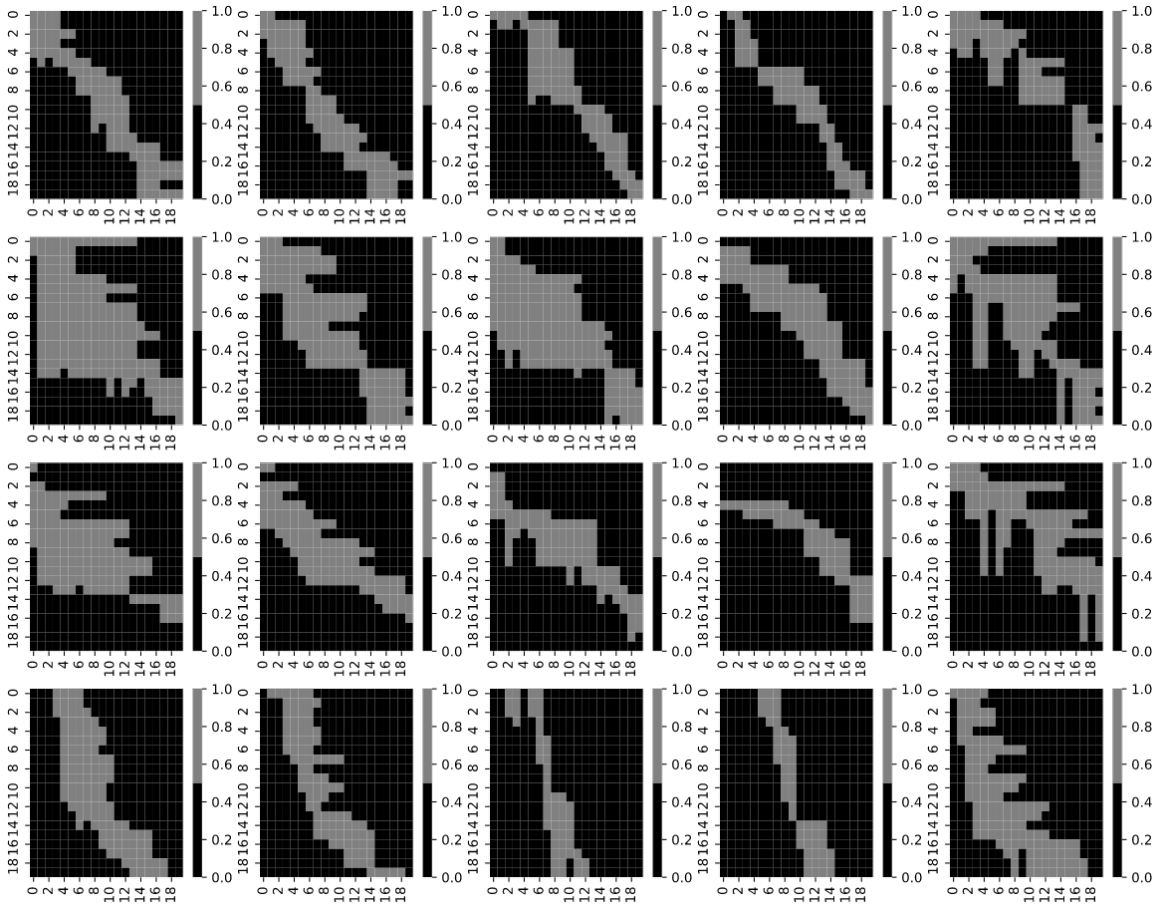

Figure 4.2 Temporary Image - Cols= R, Rows = X

Election Dimensions Our results for two election sizes have one crucial difference, looking briefly at Figure 4.3 (20 candidates, 20 voters) and Figure **??** (40 candidates, 40 voters), we immediately see, that in the larger elections *NCOP* property is much more frequent. (WIP: Jakaś hipoteza, dlaczego tak się dzieje?) Election sizes in the experiment are relatively small

when compared to the real world scenario. We can expect for this trend of increased number of *NCOP* profiles to continue for even larger elections. This result is very promising, as it strongly encourages further development and research related to the VCR domain.

(WIP: Czy podwójna linia przewy może być? innaczej akapity zlewają się za bardzo, ten akapit jest jedynie "wstawka" pomiedzy faktyczna treścią) In the subsequent paragraphs we will focus mostly on the results for smaller elections from Figure 4.3, but only because the differences between the parameters are more pronounced.

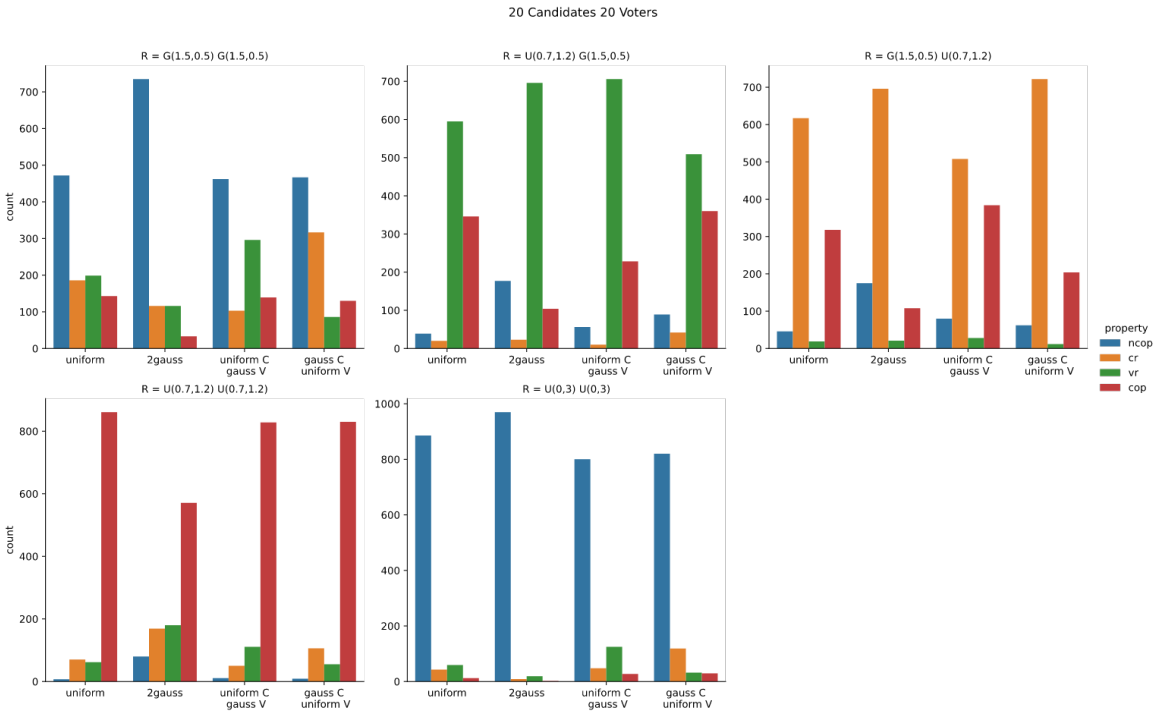

Figure 4.3 Temporary Image - Aggregated Statistics

Position Choice We will analyze the histogram for the ***Gauss Candidates, Gauss Voters*** radius model, as it exhibits the most neutral behavior. It will be our baseline scenario. We can compare position models in pairs. Starting with *Uniform Candidates, Uniform Voters* and *Asymmetric Gauss Candidates, Asymmetric Gauss Voters* models. The double Gauss model has noticeably more *NCOP* profiles. It can be easily explained by looking at the respective preference matrix heatmaps in Figure 4.2. The profiles generated from the double Gauss model have higher count of approvals, which in turn increases the likelihood of any irregularities in the approvals block structure, which is directly linked to the *NCOP* property. (WIP: lepiej wyjaśnić, mam na myśli "dziury" w blokach jedynek w macierzy)

Now turning our attention to the second pair of the position models (still for the same radius model), *Uniform Candidates, Asymmetric Gauss Voters* and *Asymmetric Gauss Candidates, Uniform Voters*. These are two perfectly symmetrical models, we will explain only the first one, and keep in mind that second one is just the reversed scenario. Looking at the heatmaps again, we notice that the blocks of approvals are skewed horizontally, which is associated with the VR property, i.e., the ones form and interval in each row of the approval matrix. For the second model, the blocks are skewed vertically, which links to the CR property.

Radius Choice From Figure 4.3 it is clear that the choice of radius can significantly reinforce or subside trends (in the profiles properties statistics) resulting from the position choice. For example, the mixed radius models, i.e., *Small Uniform Candidates, Gauss Voters* and *Gauss Candidates, Small Uniform Voters* particularly increase the VR or CR properties frequency. The core idea behind this phenomenon is exactly the same as the one described above, for the mixed position models (Uniform, Gauss), where the blocks of approvals are skewed, either horizontally or vertically, resulting in the increased number of VR or CR profiles respectively.

The striking difference between histograms for *Small Uniform Candidates, Small Uniform Voters* and *Large Uniform Candidates, Large Uniform Voters* is again related to the inconsistencies in the approval blocks, which are easily observed in the heatmaps. Smaller radius range produces much more structured profiles, hence large number of the *COP* profiles. In contrast, large radius range outputs irregular approval blocks in the profiles, increasing number of the *NCOP* profiles.

Summary (WIP: Czy brakuje akapitu podsumowującego? Powtórzenie z Election Size) This result is very promising, as it encourages further development and research related to the VCR domain.

4.3 Transforming VCR (NCOP) Profile into VR or CR

A natural question that arises from the previous experiment results, is how different are the NCOP profiles from the VR or CR ones? In other words, how much do we need to change a NCOP profile for it to become a VR / CR one. To answer this question, we design another experiment.

4.3.1 Experiment 3 — Profile Transformation

In this experiment, we partially reuse profiles generated for the previous one. We narrow down our experiment parameters to only two radius generation methods, that is, first model *Gauss Candidates, Gauss Voters* and second *Large Uniform Candidates, Large Uniform Voters*. We keep the position models unchanged, and we explore all four of them. Obviously we only pick the NCOP profiles, to be exact, we use 400 of them for each parameter pair.

Agent Deletion Parameter We consider 3 methods to transform a NCOP profile into a VR / CR one.

1. The *Candidate Deletion* method, where we search for a minimal number d, such that, after deleting d candidates from the profile it becomes a VR / CR profile.
2. The *Voter Deletion* method, which is similar to the *Candidate Deletion* method, but this time we delete d voters instead.
3. The *Candidate and Voter Deletion* method, where we combine the two previous methods, and we simultaneously delete d_c candidates and d_v voters.

Transformation Result Parameter Apart from choosing what type of agent (candidate / voter) to delete, for each profile we collect the transformation results into 3 groups. By the transformation result we mean the lowest possible cost of the transformation, i.e., the minimal number of agent deletions for a TVCR profile to become a VR / CR one.

1. The *Candidate Range*, where the cost is a number of deletions required to get a CR profile.
2. The *Voter Range*, where the cost is a number of deletions required to get a VR profile.
3. The *Candidate Range or Voter Range*, where the cost is a minimum from the previous two groups.

Results

Chapter 5

Control Problem

5.1 Overview

5.1.1 AV-CC-DV for Single Peaked (VR = CI = row C1P = $r_c = 0$)

Algorithm from Faliszewski et al. (2009)

1. kandydat p na osi, pozostali na lewo i prawo
2. wybór przeciwników (groźny kandydat):
   (a) pierwszy na lewo c_L i pierwszy na prawo c_R takich, że $score(c_?) \geq score(p)$
   (b) powtarzamy dla kolejnych lewych c_{L_i} takich, że $score(c_{L_i}) > score(c_{L_{i-1}})$
   (c) analogicznie dla prawych c_{R_i} takich, że $score(c_{R_i}) > score(c_{R_{i-1}})$
3. usuwanie głosów - zaczynając od najbliższego prawego przeciwnika c_R (dla lewych - odbicie lustrzane)
   (a) notacja v: $b(v)$ początek przedziału (lewy) i $e(v)$ koniec przedziały (prawy)
   (b) wybieramy głosujących $\widehat{V}$ takich że $\{v \in V | \, b(v) \in (p, c_R]\}$
   (c) sortujemy $\widehat{V}$ malejąco po $e(v)$ (od najbardziej prawego końca przedziału)
   (d) usuwamy $scoreDelta(p, c_R) + 1$ głosów
4. powtarzamy do skutu, przeliczając aprobaty co iterację

5.1.2 AV-CC-DV for Candidate Range = VI = column C1P = $r_v = 0$

Algorithm from Magiera and Faliszewski (2014)

1. notacja CR dla c: $v_{f(c)}$ początek przedziału (lewy/górny) i $v_{e(c)}$ koniec przedziału (prawy/dolny)
2. kandydat p
3. Głosujący - wybieramy głosujących $V' = \{v \in V | \, p \notin A(v)\}$
4. Przeciwnicy - wybieramy kandydatów $\widehat{C} = \{c \in C \setminus \{p\} | \, scoreDelta(c, p) \geq 0\}$
5. Usuwanie głosów
   (a) zaczynając od przeciwnika c_N takiego, że $v_{e(c_N)}$ jest minimalne
   (b) usuwamy $deltaScore(c_N, p) + 1$ głosów, zaczynając od końca przedziału $v_{e(c_N)}$
6. powtarzamy do skutu, przeliczając V' i $\widehat{C}$ co iterację

5.2 AV-CC-DV for the VCR profiles

Notation used in the algorithm and later in the proofs:
x_a, r_a — point and radius
$b(a) = x_a - r_a$ — range beginning (left)
$e(a) = x_a + r_a$ — range end (right)
p — candidate chosen to be a winner
$\widehat{C} = \{c \in C \setminus \{p\} | \, scoreDelta(c, p) \geq 0\}$ — opponents (dangerous candidates)
$V' = \{v \in V | \, p \notin A(v)\}$ — potential voters to delete

We use “smart” greedy strategy for voter deletion:

1. We pick a leftmost opponent c_L that is the one with the minimal $b(c_L)$
2. We consider only voters who approve of c_L that is, $\widehat{V} = \{v \in V' | \, c_L \in A(v)\}$
3. We need to delete $deltaScore(c_L, p) + 1$ rightmost voters from $\widehat{V}$ that is the ones with the maximal $e(v)$, if it’s not possible we fail
4. We repeat the above steps until we succeed, recalculating $\widehat{C}$ and $\widehat{V}$ after each iteration

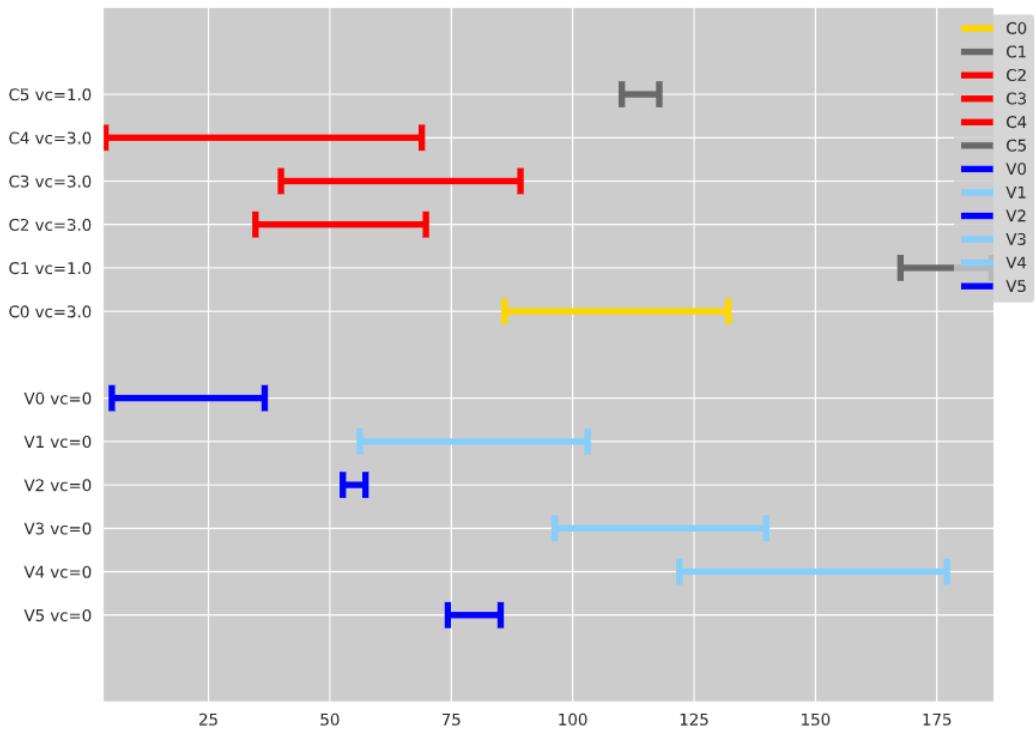

Figure 5.1 Temporary Image - VCR !C1P Election
yellow — candidate p
red — opponents
lightblue — voters whitelist
darkblue — possible voters to delete
grey — agents that can be ignored

5.2.1 Sketch Proof - CR case

We will show how the VCR algorithm is equivalent to the CR one from Magiera and Faliszewski (2014). Having voters in CR order, that is, they are still represented by a point x_v on the $\mathbb{R}$ line but have radius 0. Example in Figure 5.2.

Choosing opponents

In the CR algorithm, we choose candidates to process, ordered by $v_{e(c)}$ ascending. We can see that it is equivalent to picking the leftmost $(min(b(c)))$ candidate from the set of opponents $\widehat{C}$ in the VCR algorithm, because it will also have the minimal $v_{e(c)}$.

There is a special case of "contained candidate", for which the above statement is not true. In Section 5.2.3, we explain in detail why it is irrelevant for the overall correctness of the algorithm.

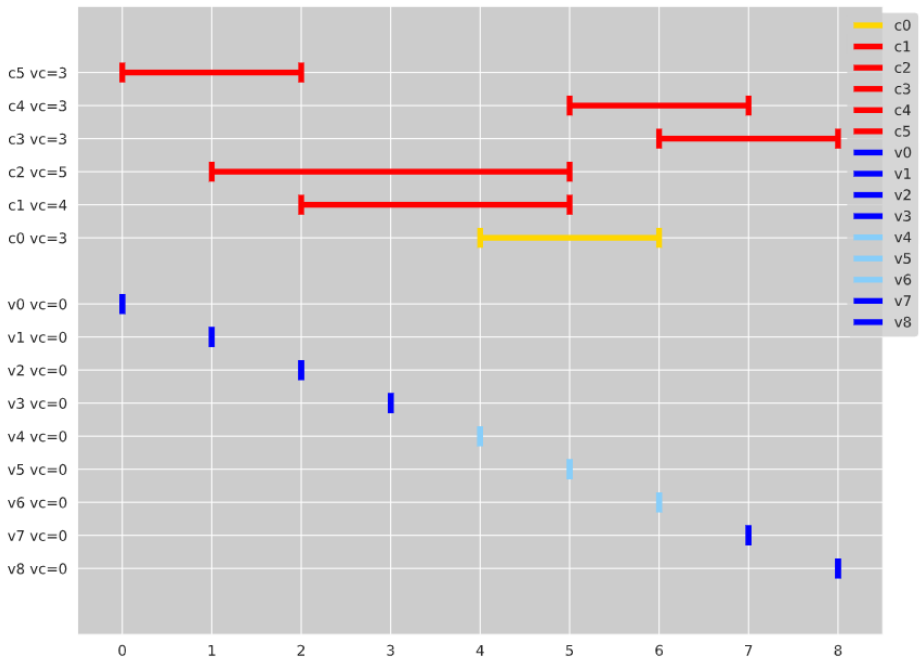

Figure 5.2 Temporary Image - CR Election

Voter deletion

Now, once we now how to pick an opponent c_L, we will show that choosing voters is equivalent as well. Notion of the beginning/end of the interval in the CR order is equivalent to the left/right end of the range in the VCR setting. Both algorithms try to delete $scoreDelta(p, c_L) + 1$ voters, CR one starts deleting voters from the end of interval, while VCR does virtually the same by deleting the rightmost voter first ($max(e(v))$).

5.2.2 Sketch Proof - VR case

We will show how the VCR algorithm is equivalent to the VR one from Faliszewski et al. (2009). Having candidates in VR order, that is, they are still represented by a point x_c on the $\mathbb{R}$ line but have radius 0. Example in Figure 5.3.

Choosing opponents

VR algorithm picks dangerous candidates to the left and right of candidate p, however later it proceeds only with defeating the right ones. Once finished with them, it does the "mirror flip" of the universe to process candidates on the left side. Even though the VCR algorithm picks leftmost candidate, we will show it is equivalent to the VR strategy under some basic assumptions.

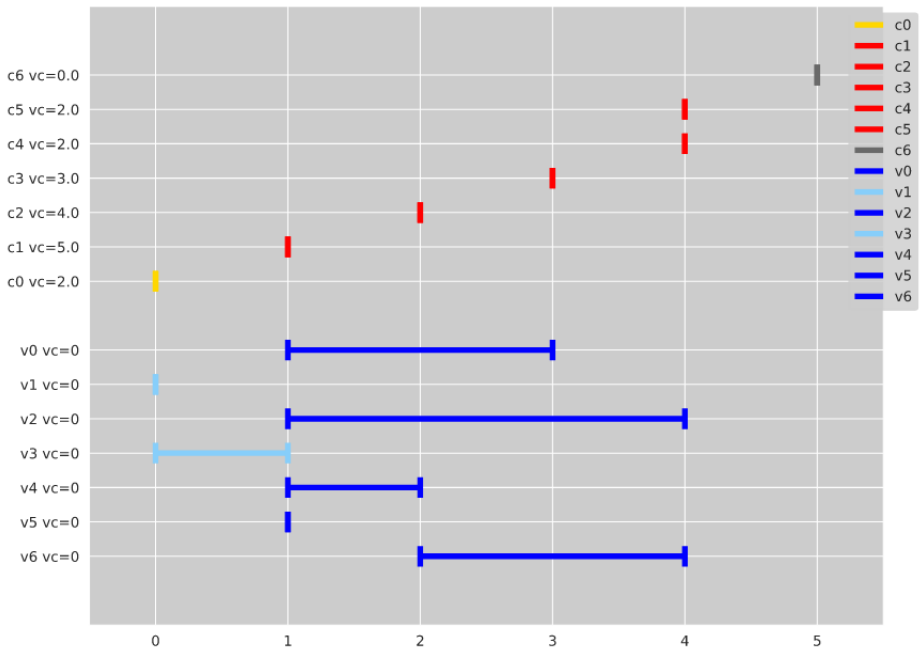

Figure 5.3 Temporary Image - VR Election

Following example in Figure 5.3, let's assume that candidate p always will be the first candidate in the VR order, which is equivalent to being the leftmost in the VCR order. Then we can see that the VCR algorithm strategy of choosing the leftmost candidate as an opponent will be the same as the VR algorithm choosing the first candidate on the right side of p. In the example it would be candidate c_1.

This assumption is correct because, having candidates in order $[c_L, p, c_R]$, if there was a voter v who approved of c_L and c_R it would also have to approve of p (because of the VR property), so we could not delete v anyway.

Now, expanding on the above, ...

(WIP: Faktycznie taki dowód nie jest w 100% przekonujący Mam 2 możliwości: 1) Wprowadzić krork "universe flip" do algorytmu i wtedy na pewno działa. 2) Zmienić argumentację i pokazać, że wybieranie najdalszego kandydat od p (najbardziej lewy po lewej stronie) też jest w porządku)

Voter deletion

The reasoning here is very similar to the one in the voter deletion part of the CR proof. Both algorithms try to delete $scoreDelta(c, p) + 1$ voters. VR algorithm starts deleting voters from the end of interval $(max(e(v)))$. VCR one does virtually the same by deleting the rightmost voter first.

5.2.3 Sketch Proof - VCR (not C1P) case

Now we will show why our solution is correct in the general VCR setting, even when the profile does not have VR/CR property, example in Figure 5.1. The proof is split into two stages, similarly to the previous ones. But first we will explain one special case of the election, that is worth examining.

Special case - contained candidates - alternative explanation

In the VCR model it is possible that some candidates have their interval fully contained in the other's interval. Every voter who would approve the smaller candidate would also approve the larger one, however not vice versa. For this case, our strategy of choosing voters to delete might seem suboptimal, at first glance.

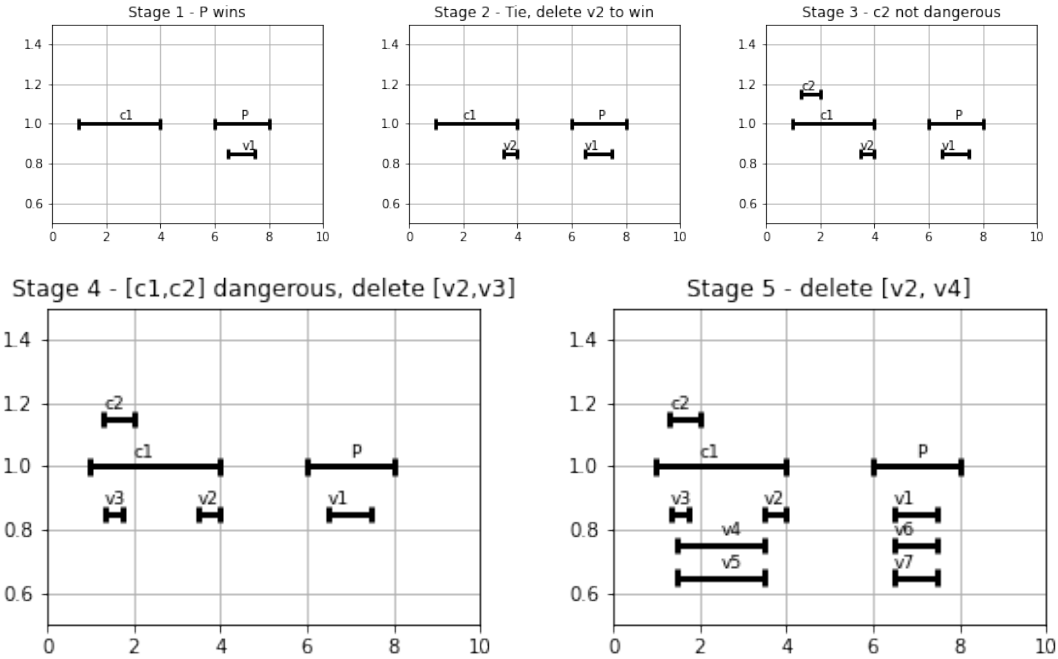

Figure 5.4 Temporary Image - Special Case

We explain the issue, starting with the example of the most basic election in Figure 5.4. Stages 1 through 3 are trivial, in the subsequent stages we make the election more and more complex by adding new agents. Turning our attention to Stage 4, now both candidates c_1 and c_2 are dangerous, with scores 2 and 1 respectively. Candidate p needs to defeat c_2 by one vote, and c_1 by two votes. In Stage 5 we add multiple more voters ...

The core idea to solve our issue is understanding what types of the new votes we can add. Let's consider only the voters who approve of c_1 and/or c_2.

1. A voter who approves only c_1 and not c_2, in this case, $scoreDelta(p, c_1)$ would increase by 1, and $scoreDelta(p, c_2)$ would stay the same.

2. A voter who approves both c_1 and c_2, this time, $scoreDelta(p, c_1)$ and $scoreDelta(p, c_2)$ would increase equally by 1.

Proposition 3. *Having external candidate c_E and another internal one c_I, which is fully contained inside external's range.*

Let:

1. $K' = \{v \in \widehat{V} \mid v \in A(c_E) \wedge v \notin A(c_I)\}$.

2. $|K'| = scoreDelta(p, c_E) - scoreDelta(p, c_I)$.

3. $d(c) = score(c) - score(p) + 1$, *i.e., how many voters we need to delete for p to defeat c.*

Then $|K'| \geq 0$ is always true, and the size of set K' will be equal to the $d(c_E) - d(c_I)$

Proof. tbc. □

Special case - contained candidates

In the VCR model it is possible that some candidates have their interval fully contained in the other's interval. Every voter who would approve the smaller candidate would also approve the larger one, however not vice versa. For this case, our strategy of choosing voters to delete might seem suboptimal, at first glance.

The example from Figure 5.5 illustrates why we do not have to worry about this special case. Every contained voter will have as much or less total votes than the larger candidate, he/she might not even be a dangerous candidate at all. Now we see that in the process of defeating the larger candidate we will automatically defeat the smaller one.

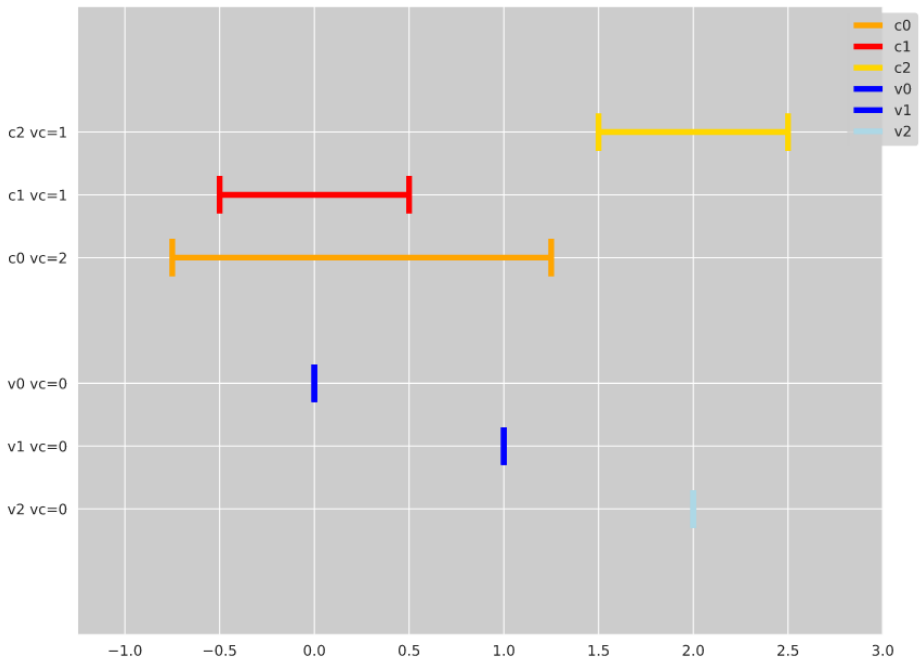

Figure 5.5 Temporary Image - Special Case

Choosing opponents

We consider only dangerous candidates (opponents), that is the ones with score higher or equal to the score of p. We process opponents one by one, starting with the one who has the leftmost beginning of the range, i.e., $min(b(c))$. After each iteration, we recompute the set of dangerous candidates. Thanks to this we are sure that all the steps we perform are necessary to find a solution (if it exists), hence optimal.

Voter deletion

Having currently processed opponent c_L, we consider only the voters who approve of him/her and do not approve of p, that is the voters whose range beginning is inside the candidate's range $\{v \in V' \mid b(v) \in [b(c_L), e(c_L)]\}$. We need to delete $scoreDelta(c_L, p) + 1$ voters to defeat the opponent. By picking the voters in order of the rightmost end of the range, i.e., $v_R = max(e(v))$, we try to maximize the number of other candidates who might be in the range $[b(v_R), e(v_R)]$. We don't have to worry about any candidate to the left, because of our leftmost candidate picking strategy, they are already defeated. This strategy ensures the solution is optimal if it exists.

5.3 AV-CC-AV for the VCR profiles

Our strategy is based on the solution from Magiera and Faliszewski (2014).

1. Add all $v \in W$ to the election
2. Is p the winner?
3. Try to delete $k' = |W| - k$ voters

Chapter 6

Evaluation

6.1 Conclusions

6.2 Future Work

Bibliography

Arrow, K. (1951). *Social Choice and Individual Values*. Wiley: New York.

Barberà, S. and Moreno, B. (2011). Top monotonicity: A common root for single peakedness, single crossing and the median voter result. *Games and Economic Behavior*, 73(2):345–359.

Black, D. (1958). *The theory of committees and elections*. University Press, Cambridge.

Booth, K. S. and Lueker, G. S. (1976). Testing for the consecutive ones property, interval graphs, and graph planarity using pq-tree algorithms. *Journal of Computer and System Sciences*, 13(3):335 – 379.

Brams, S. and Fishburn, P. (1983). *Approval Voting*. Southgate Publishers.

Brandt, F., Brill, M., Hemaspaandra, E., and Hemaspaandra, L. A. (2015). Bypassing combinatorial protections: Polynomial-time algorithms for single-peaked electorates. *Journal of Artificial Intelligence Research*, 53:439–496.

Elkind, E. and Lackner, M. (2015). Structure in dichotomous preferences. *CoRR*, abs/1505.00341.

Elkind, E., Lackner, M., and Peters, D. (2016). Preference restrictions in computational social choice: Recent progress. In *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence*, IJCAI'16, page 4062–4065. AAAI Press.

Faliszewski, P. (2021). An analysis of approval-based committee rules for 2d-euclidean elections.

Faliszewski, P., Hemaspaandra, E., Hemaspaandra, L. A., and Rothe, J. (2009). The shield that never was: Societies with single-peaked preferences are more open to manipulation and control. *CoRR*, abs/0909.3257.

Faliszewski, P., Skowron, P., Slink, A., and Talmon, N. (2017). *Multiwinner Voting: A New Challenge for Social Choice Theory*. AI Access.

Lackner, M. and Skowron, P. (2020). Approval-based committee voting: Axioms, algorithms, and applications.

Magiera, K. and Faliszewski, P. (2014). How hard is control in single-crossing elections? *Frontiers in Artificial Intelligence and Applications*, 263:579–584.

Mirrlees, J. A. (1971). An Exploration in the Theory of Optimum Income Taxation. *Review of Economic Studies*, 38(2):175–208.

Roberts, K. W. (1977). Voting over income tax schedules. *Journal of Public Economics*, 8(3):329–340.

Skowron, P., Yu, L., Faliszewski, P., and Elkind, E. (2015). The complexity of fully proportional representation for single-crossing electorates. *Theoretical Computer Science*, 569:43–57.

Walsh, T. (2007). Uncertainty in preference elicitation and aggregation. In *Proceedings of the 22nd National Conference on Artificial Intelligence - Volume 1*, AAAI'07, page 3–8. AAAI Press.